

Chapitre 1 : Rappels sur les systèmes d'exploitation

I. Introduction

- Un SE gère le matériel et fait le lien avec les applications.
- Il peut être vu comme :
 - **Machine virtuelle** : chaque utilisateur a l'impression d'avoir son propre ordinateur.
 - **Gestionnaire de ressources** : partage équitable des ressources entre programmes.

II. Fonctions principales d'un SE

1. **Gestion de la mémoire** : allocation, libération, suivi et mémoire virtuelle.
2. **Gestion des processus et du CPU** : création, suspension, reprise, ordonnancement.
3. **Gestion des fichiers** : création, suppression, organisation, accès via primitives.
4. **Gestion des E/S** : interface uniforme, pilotes spécifiques aux périphériques.
5. **Protection et sécurité** : isolement des processus, contrôle des accès et des droits.

III. Historique des SE

1. **1^{re} génération (1945–1955)** : pas de SE, programmes en langage machine, un seul programme à la fois.
2. **2^e génération (1955–1965)** : monoprogrammation, traitement par lots.
3. **3^e génération (1965–1980)** : multiprogrammation, temps partagé, spooling.
4. **4^e génération (1980–1990)** : PC, réseaux, temps réel.
5. **5^e génération (1990–aujourd'hui)** : multiprocesseurs, systèmes embarqués (Android, supercalculateurs).

IV. UNIX / LINUX

- **UNIX** : créé par Thompson et Ritchie (1960), réécrit en C, standardisé POSIX.
- **LINUX** : créé par Linus Torvalds (1991), open-source, distributions populaires : Ubuntu, Debian, Fedora.

V. Caractéristiques

- Multiutilisateur et multitâche.
- Processus hiérarchiques et création dynamique (`fork()`).
- Communication IPC : sémaphores, tubes, signaux, sockets.
- Fichiers hiérarchiques et périphériques vus comme fichiers.
- Mémoire virtuelle, protection par mots de passe et droits, super-utilisateur (`root`).

VI. Interaction utilisateur

- Via **Shell** : lancer programmes, gérer fichiers, scripts.
- SE fournit des **bibliothèques d'appels système**.

VII. Interface SE – Matériel

- **Interruptions** : matérielles (horloge) et logicielles (division par zéro).
- **Appels système** : passage mode utilisateur → mode superviseur → exécution → retour.
- Types : gestion processus, fichiers/périphériques, maintenance.

VIII. Noyaux

1. **Monolithique** : tous les services dans le noyau, rapide mais moins sûr (ex : UNIX).
2. **Micro-noyau** : noyau minimal, services en mode utilisateur, plus sûr mais plus lent (ex : RTLinux).

IX. Implémentation

- Historiquement en Assembleur, aujourd'hui en **C** (UNIX/Linux, Windows NT).
- Avantages du C : modulaire, portable, plus facile à maintenir.

Partie 2 : Système de Gestion de Fichiers (SGF)

I. Introduction

- **Fichier** : unité logique de stockage (nom, emplacement, attributs).
- **Répertoire** : organise les fichiers.
- **Système de fichiers** : gère création, suppression, accès, protection.

II. Catégories de fichiers

- **Ordinaires** : données utilisateur.
- **Répertoires** : regroupent fichiers/répertoires.
- **Spéciaux** : périphériques (ex : `/dev/sda`).

III. Organisation de l'espace disque

- Données stockées en **blocs** (512 à 2048 octets).
- Chaque fichier = ensemble de blocs numérotés.

IV. Techniques d'allocation

1. **Contiguë** : blocs successifs, accès rapide, risque de fragmentation.
2. **Chaînée** : blocs dispersés.
 - Liste chaînée : chaque bloc pointe sur le suivant.

- Liste chaînée indexée (FAT) : table en mémoire pour suivre les blocs.
- I-Node (UNIX/Linux) : attributs + adresses blocs, répertoire pointe sur I-Node.

V. Protection des fichiers

- **Contrôle d'accès** : USER, GROUP, OTHER.
- Exemple : `-rwxr-xr--` (lecture, écriture, exécution pour USER, lecture/exécution pour GROUP, lecture pour OTHER).

VI. Partage de fichiers

1. **Lien physique (hard link)** : même I-Node, fichier supprimé si tous les liens effacés.
 - Commande : `ln fichier_original nouveau_nom`.
2. **Lien symbolique (symlink)** : fichier spécial contenant chemin du fichier cible, I-Node différent.
 - Commande : `ln -s fichier_original lien_symbolique`.
 - Si le fichier d'origine est supprimé → lien cassé.

Exemple pratique :

- Créer fichier `fic1`.
- Lien physique `fic2` → même I-Node que `fic1`.
- Lien symbolique `fic3` → I-Node différent.
- Modifier `fic2` → affecte `fic1`.
- Modifier `fic3` → ne change pas `fic1`.
- Supprimer `fic1` → `fic2` reste valide, `fic3` devient cassé.

Chapitre 2 : Système de Gestion de Fichiers (SGF)

1. Concepts de base

1.1 Fichier

- Unité logique de stockage pour conserver des données (texte, images, programmes...).
- Attributs : nom, extension, date/heure, taille, droits d'accès.
- Opérations principales : création, lecture, écriture, positionnement, troncature, suppression, ajout, renommage, copie, ouverture, fermeture.
- Exemple :

```
$ cat > fichier
```

```
abcd
```

```
# crée un fichier nommé "fichier" contenant "abcd"
```

1.2 Système de gestion de fichiers (SGF)

- Module du SE gérant stockage, accès et manipulation des fichiers.
- Rôle : interface entre utilisateur, programmes et matériel.
- Fonctions : stockage hiérarchique, transparence du périphérique et du format (FAT, NTFS, ext4...).

1.3 Répertoire

- Structure pour organiser les fichiers.
- Répertoire = fichier spécial contenant des entrées (nom + attributs + pointeur vers blocs).
- Chemins : absolus (/home/user/fichier) ou relatifs (depuis répertoire courant).
- Différence fichier/répertoire : bit d'attribut spécifique.

2. Rôles du SGF

- Interface simple pour manipulation des fichiers (création, copie, suppression...).
- Organisation et allocation de l'espace disque.
- Suivi de l'espace libre.
- Gestion multi-utilisateur.
- Outils de diagnostic et récupération.

3. Gestion de l'espace disque

3.1 Clusters

- Données stockées en blocs regroupés en clusters.
- Taille influence performance et gaspillage :
 - Grand cluster → moins de gestion, plus de gaspillage
 - Petit cluster → moins de gaspillage, accès plus lent

3.2 Techniques d'allocation

1. **Allocation contiguë** : blocs successifs, rapide, mais fragmentation et difficile à prévoir la taille.
2. **Allocation chaînée** : blocs reliés par pointeurs, flexible, accès séquentiel lent, corruption possible.
3. **Allocation indexée** : pointeurs centralisés (table)
 - FAT (MS-DOS/Windows)
 - NTFS (Windows, MFT, noms longs, sécurité)
 - I-Node (UNIX/Linux, ext3/ext4)
 - Contient attributs, blocs directs (10), indirections (simple/double/triple) pour fichiers volumineux.

3.3 Gestion de l'espace libre

- **Bitmap** : 0 = libre, 1 = occupé

- **Liste doublement chaînée (FAT)** : blocs libres reliés en liste

4. Autres systèmes et critères

- Comparaison : taille max disque/fichier, noms, vitesse, fragmentation, tolérance aux erreurs, sécurité.

5. Étude de cas Linux

5.1 Catégories de fichiers

- **Ordinaires (-)** : texte, code, multimédia, exécutable
- **Répertoires (d)** : organisent fichiers/sous-dossiers
- **Spéciaux** : périphériques
 - Bloc : /dev/sda, /dev/sdb
 - Caractère : /dev/tty, /dev/null
- **Liens (symboliques/physiques)**
 - Hard link : partage inode, même partition, fichier supprimé si tous les liens effacés
 - Symlink : inode propre, peut pointer sur autres partitions, devient cassé si fichier d'origine supprimé

5.2 Partage de fichiers

- Permet cohérence et éviter duplication via hard link ou symlink.
- Commandes : `ln`, `ln -s`, `rm`.

5.3 Implémentation pratique

- **Formatage** : `mkfs`, initialise Ext4 ou autre SGF sur une partition.
- **Ext4** : successeur d'Ext3/Ext2, extents pour gros fichiers, partitions jusqu'à 1 Exaoctet.
- **Montage (mount)** : attache une partition à un point de l'arborescence (/home, /mnt...), abstraction via VFS.
- **Administration** :
 - `fsck` → vérifie et répare le SGF
 - `tune2fs` → paramètres avancés Ext4 (journalisation, fréquence vérifications)

Chapitre 3 : Gestion des processus

1. Rappels

- **Algorithme** : description abstraite d'un comportement pour résoudre un problème.
- **Programme** : traduction d'un algorithme en langage exécutable.
- **Processeur** : exécute le programme.
- **Ressource** : élément nécessaire à un processus (CPU, mémoire, fichiers...).

- **Job / Travail** : ensemble d'étapes exécutables par le système, identifié par un utilisateur.

2. Processus

2.1 Exécution

- **Séquentielle** : instructions exécutées dans un ordre strict.
- **Parallèle** : certaines instructions peuvent s'exécuter simultanément.
- **Processus séquentiel** : exécution ordonnée d'un programme.

2.2 Différence Programme vs Processus

- Programme : statique
- Processus : dynamique, instance d'un programme en cours d'exécution.

2.3 États principaux

- **Prêt (Ready)** : attend le CPU
- **Actif (Running)** : en exécution
- **Bloqué (Waiting)** : attend une ressource ou un événement

2.4 Transitions

- Changement d'état : disponibilité de ressources, arrivée d'événements, priorité, E/S, fin de processus.
- Structures : files pour prêt et bloqué, tableau pour actif.

2.5 Environnement d'un processus

1. Image mémoire

- Text Segment : code exécutable
- Data Segment : données initialisées/non initialisées
- Heap : mémoire dynamique
- Stack : variables locales et adresses retour
- Variables d'environnement, bibliothèques partagées

2. Image processeur

- Registres (PC, SP, état, généraux)
- Contexte d'exécution sauvegardé/restauré via PCB

2.6 PCB (Process Control Block)

- Contient :
 - Contexte d'exécution (registres, pile, mémoire)
 - Informations de gestion (PID, priorité, état, relations père/fils, fichiers ouverts...)
- Permet **context switching** : sauvegarde/restauration d'un processus pour multitâche.

2.7 Tables des processus

- Ensemble des PCB stocké en mémoire par le SE.
- Non accessible aux processus.

2.8 Opérations sur processus

- Création, destruction, blocage, réveil, activation, suspension.
- **Création** : allocation PCB, PID, initialisation mémoire et pile.
- **Destruction** : libération des ressources, PCB réutilisable, PID non réutilisable.

3. Processus UNIX/Linux

3.1 Arguments d'un programme

- `argc` : nombre d'arguments
- `argv` : tableau des arguments
- `arge` : tableau des variables d'environnement

3.2 Variables d'environnement

- Syntaxe : `NOM=VALEUR`
- Principales sous Linux : `HOME`, `PATH`, `PWD`, `USER`, `SHELL`, `TERM`
- Manipulation en C : `getenv()`, `putenv()`, `setenv()`, `unsetenv()`

3.3 Appels système POSIX

1. **Création : `fork()`**
 - Du père vers le fils, PID unique, copie du PCB (copy-on-write pour mémoire)
2. **Terminaison : `exit(status)`**
 - Status 0 = normale
3. **Attente du fils : `wait()` / `waitpid()`**
 - Évite les zombies
 - Décodage du status via macros `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`...
4. **Mise en pause : `sleep(nbSec)`**

3.3.2 Lancement de programmes : `exec()`

- Remplace l'image mémoire du processus courant par un nouveau programme
- Variantes : `execl`, `execlp`, `execv`, `execve`, `execle`, `execvp`
- Paramètres : arguments et variables d'environnement
- Après `exec`, le code courant ne s'exécute plus sauf erreur.

II. Threads

- **Thread** : séquence d'instructions indépendante, partage la mémoire et les ressources du processus.

- Différence avec processus : un processus possède sa propre mémoire, un thread partage la mémoire du processus.

Opérations sur threads

1. **Création** : `pthread_create()`
2. **Attente de terminaison** : `pthread_join()`
3. **Identification** : `pthread_self()`
4. **Terminaison** : `pthread_exit()`

Exemple

- Plusieurs threads peuvent s'exécuter en parallèle et partager les ressources du processus, améliorant performances et réactivité.

Chapitre 4 : Les tubes (Pipes)

1. Notions de base

1.1 Descripteurs de fichiers

- Un **descripteur de fichier** est un identifiant utilisé par un processus pour accéder à une ressource d'E/S.
- Créé lors de l'ouverture d'un fichier.
- Utilisé avec `read()`, `write()`, `close()`.
- Toutes les E/S passent par un **buffer** en mémoire.

1.2 Flots d'entrée / sortie

- **Flux d'entrée** : clavier, fichiers (lecture).
- **Flux de sortie** : écran, fichiers (écriture).
- Chaque flux repose sur un descripteur.
- À la création d'un processus, trois flux standards existent :
 - entrée standard
 - sortie standard
 - sortie d'erreur

2. Redirections

Les **redirections** permettent de modifier la source ou la destination des flots standards.

2.1 Redirection de la sortie

- `>` : écrase le fichier.
- `>>` : ajoute à la fin.
- Exemple :
`ls > listing` → la sortie de `ls` est envoyée dans `listing`.

2.2 Redirection de l'entrée

- `<` : la commande lit depuis un fichier au lieu du clavier.
- Exemple :
`sort < listing`

2.3 Redirection des erreurs

- `2>` : redirige la sortie d'erreur.
- `2>>` : ajoute les erreurs à la fin d'un fichier.
- Exemple :
`ls fiche 2> erreur`

3. Tubes de communication (Pipes)

3.1 Définition

- Un **pipe** est un canal de communication **unidirectionnel**.
- Fonctionne en **FIFO** (First In, First Out).
- Il redirige la sortie d'un processus vers l'entrée d'un autre.

Exemple shell

```
cat f1 f2 | grep bon | wc -l
```

- `cat` écrit dans un pipe
- `grep` lit, filtre, réécrit
- `wc -l` compte les lignes

4. Types de tubes

4.1 Pipes anonymes

- Communication entre processus **parent-fils**.
- Créés par `pipe()`.
- Deux descripteurs :
 - `desc[0]` : lecture
 - `desc[1]` : écriture
- Taille limitée (~4 Ko).

Fonctionnement

1. Création du pipe
2. `fork()`
3. Fermeture des descripteurs inutiles
4. Échange via `read()` / `write()`

Exemple (résultat)

Le fils reçoit le message envoyé par le père :

J'ai bien reçu votre message: bonjour

Avantages

- Simple
- Rapide
- Gestion automatique
- Idéal père ↔ fils

Inconvénients

- Unidirectionnel
- Parenté obligatoire
- Données non persistantes

4.2 Pipes nommés (FIFO)

Définition

- Tube identifié par un **fichier spécial**.
- Communication entre **processus indépendants**.
- FIFO persistant dans le système de fichiers.

Création

- En shell : `mkfifo`, `mknod`
- En C : `mkfifo()`

Utilisation

- `open()` : ouverture
- `read()` / `write()` : communication
- `close()` : fermeture
- `unlink()` : suppression

Exemple

- Un processus écrit dans le FIFO.
- Un autre lit le message.
- Résultat :

fin d'écriture

j'ai reçu message

Avantages

- Communication entre processus non liés
- Persistant

- Manipulable comme un fichier

Inconvénients

- Moins performant
- Doit être créé et supprimé explicitement
- Risque de blocage si mauvaise synchronisation

Conclusion

- Les **pipes** sont un mécanisme IPC fondamental sous Unix/Linux.
- **Pipes anonymes** : rapides, simples, parent–fils.
- **Pipes nommés** : plus flexibles, processus indépendants.
- Très utilisés avec les **redirections** et le **shell** pour chaîner des commandes.

Chapitre 5 : IPC par envoi de messages & mémoire partagée

1. Rappels sur l'IPC System V

Définition

Les **IPC (Inter-Process Communication)** permettent aux processus Unix/Linux d'échanger des données et de partager des ressources.

Intérêts

- Communication entre processus **indépendants**
- Synchronisation de processus concurrents
- Partage efficace de ressources

Types d'IPC System V

- **Files de messages** : `msgget`, `msgsnd`, `msgrcv`, `msgctl`
- **Mémoire partagée** : `shmget`, `shmat`, `shmdt`, `shmctl`
- **Sémaphores** (pour la synchronisation)

Clé IPC

- Chaque objet IPC est identifié par une **clé (key_t)**.
- Générée avec `ftok()`.
- Les processus doivent partager la même clé pour accéder au même objet.

2. Modes d'échange de données

Mode synchrone

- Émetteur et récepteur doivent être prêts simultanément.
- Risque de blocage.

Mode asynchrone

- L'émetteur envoie sans attendre.
- Plus flexible et plus performant.
- Utilisé par les **files de messages**.

3. Communication par files de messages

3.1 Principe

- Une **file de messages** est une structure gérée par le noyau.
- Les messages sont stockés jusqu'à leur lecture.
- Communication **asynchrone** et **persistante**.

Caractéristiques

- Messages structurés (type + contenu)
- Ordre FIFO avec gestion de priorités
- Plusieurs processus peuvent lire/écrire

Structure d'un message

```
struct msgbuf {  
    long mtype;    // type (priorité / catégorie)  
    char mtext[ ]; // contenu  
};
```

3.2 Création et accès

- `msgget()` : crée ou récupère une file
- Clé partagée entre processus

3.3 Envoi d'un message

- `msgsnd()`
- Modes bloquant ou non bloquant (`IPC_NOWAIT`)
- Possibilité de tronquer (`MSG_NOERROR`)

3.4 Réception d'un message

- `msgrcv()`
- Sélection par type :
 - type exact
 - premier message disponible
 - priorité (type négatif)

3.5 Gestion de la file

- `msgctl()` :
 - `IPC_STAT` : infos
 - `IPC_SET` : permissions
 - `IPC_RMID` : suppression

Exemple classique

- Le fils envoie un message
- Le père le reçoit
- La file est supprimée

4. Mémoire partagée

4.1 Définition

- Un **segment de mémoire partagée** est un espace mémoire commun à plusieurs processus.
- Accès direct aux données, **sans copie via le noyau**.

Avantages

- Très rapide
- Économie de mémoire
- Idéal pour de gros volumes de données

Inconvénients

- Pas de protection automatique
- Risques d'accès concurrents
- Nécessite une synchronisation explicite

5. Gestion d'un segment de mémoire partagée

Étapes principales

1. Création du segment : `shmget()`
2. Attachement : `shmat()`
3. Lecture / écriture directe
4. Détachement : `shmdt()`
5. Suppression : `shmctl(IPC_RMID)`

Appels système

- `shmget()` : crée ou récupère un segment
- `shmat()` : attache le segment
- `shmdt()` : détache le segment
- `shmctl()` : contrôle / suppression

6. Exemple père–fils avec mémoire partagée

- Le fils écrit une chaîne dans la mémoire partagée.
- Le père attend puis lit le contenu.
- Communication rapide et directe.

Résultat :

Écriture en mémoire partagée

Contenu de la mémoire partagée : section_systèmes informatiques

7. Problèmes d'absence de synchronisation

Problèmes possibles

- Conditions de course
- Écrasement de données
- Lecture incohérente
- Résultats incorrects

Exemple

Deux processus incrémentent un compteur partagé :

- Résultat attendu : 2 000 000
- Résultat réel : valeur incorrecte

Cause :

- Accès simultanés non protégés

Solution

- Utiliser des **sémaphores**, mutex ou verrous pour protéger l'accès.

Conclusion générale

- **Files de messages** :
 - Communication asynchrone
 - Structurée
 - Persistante
 - Plus sûre, mais plus lente
- **Mémoire partagée** :
 - Très rapide
 - Pas de copie
 - Risquée sans synchronisation

Choix du mécanisme :

- Peu de données, échanges structurés → files de messages
- Gros volumes, performances critiques → mémoire partagée + synchronisation

Chapitre 6 : Les Signaux en Communication Inter-Processus (IPC)

1. Définition

Les **signaux** sont des interruptions logicielles permettant de notifier un processus qu'un événement s'est produit.

Ils servent à :

- Gérer des **interruptions**,
- **Synchroniser** des processus,
- Signaler des **erreurs système**.

Ils assurent une communication **asynchrone** avec une quantité d'information limitée.

2. Caractéristiques des signaux

- Notification envoyée par le **noyau** ou un **autre processus**.
- Ne transporte pas de données (sauf **sigqueue**).
- Peut être **capturé**, **ignoré** ou **traité par défaut**.
- Chaque signal possède :
 - Un **nom** (**SIGINT**, **SIGTERM**, ...),
 - Un **numéro** (1 à **NSIG**),
 - Un **handler** (gestionnaire).
- Liste des signaux : **kill -l**.

3. Utilisation des signaux

- **Terminaison** : **SIGTERM**, **SIGKILL**
- **Synchronisation** : **SIGUSR1**, **SIGUSR2**
- **Erreurs** : **SIGSEGV**, **SIGBUS**, **SIGFPE**
- **Clavier** : **SIGINT** (CTRL+C), **SIGTSTP** (CTRL+Z)
- **Rechargement config** : **SIGHUP**

Traitements par défaut

1. Terminaison
2. Terminaison avec core dump
3. Signal ignoré
4. Suspension
5. Reprise (continuation)

Signal 0 : sert uniquement à vérifier l'existence d'un processus.

4. Gestion des signaux

Émetteurs

- **Système** : erreurs (division par zéro, violation mémoire).
- **Processus utilisateur** : coordination.
- **Utilisateur** : clavier (CTRL+C, CTRL+Z).

Récepteurs

- Processus utilisateur (avant-plan / arrière-plan),
- Processus fils,
- Démons.
- ✗ Noyau (PID 0) et processus système protégés.

États

- **Pending** : en attente,
- **Delivered** : reçu et traité.

Prise en compte

- **SIG_DFL** : action par défaut,
- **SIG_IGN** : signal ignoré,
- Handler personnalisé.

Signaux spéciaux (non interceptables)

- **SIGKILL**, **SIGSTOP**, **SIGCONT**.

5. Manipulation des signaux

a) Envoi : **kill()**

- Envoie un signal à un processus ou groupe.
- **kill(pid, 0)** teste l'existence d'un processus.
- En ligne de commande : **kill -s <signal> <PID>**.

b) Interception : **signal()** et **trap**

- **signal()** : définir un handler en **C**.
- **trap** : intercepter un signal en **shell**.
- Le handler est exécuté à la réception du signal, puis le processus reprend.

c) Attente : **pause()**

- Bloque le processus jusqu'à un signal.
- Retourne **-1** avec **errno = EINTR**.
- Ne permet pas de savoir quel signal a réveillé le processus.

6. Exemples importants

- **Synchronisation père/fils** : le fils envoie **SIGUSR1**, le père attend avec **pause()** et traite chaque requête.
- **Masquage temporaire** : **SIGINT** ignoré pendant 15 secondes puis rétabli.
- **Mot de passe avant CTRL+C** : interruption autorisée uniquement après validation.
- **Erreur arithmétique** : division par zéro déclenche **SIGFPE**, intercepté par un handler.

Conclusion

Les signaux sont un mécanisme **simple et rapide** de notification et de synchronisation entre processus.

Ils sont efficaces mais limités, et sont souvent combinés avec d'autres mécanismes IPC pour des échanges plus complexes.

MAGUEMOUN SAMY